

Estado del Arte Pasantía de Investigación

Modelo de lenguaje asistido por base de conocimiento para la evaluación técnica de requisitos de software

Carlos David Rincones Solano

1. Introducción

La ingeniería de requerimientos es una fase crítica dentro de la ingeniería de software, caracterizada principalmente por la ambigüedad del lenguaje natural, la alta dependencia del juicio humano y la dificultad de garantizar calidad, trazabilidad y cumplimiento con los estándares (ISO/IEC/IEEE 29148) en sí dicha fase se encarga de identificar, analizar, documentar y gestionar las necesidades y expectativas de los interesados (stakeholders) para un sistema de software, su objetivo principal es traducir ideas y objetivos de negocio en especificaciones claras, consistentes y sin ambigüedades que guíen el desarrollo, asegurando que el producto final cumpla exactamente con lo que los usuarios esperan.

En este contexto los LLMs son modelos de inteligencia artificial que hoy en día están en auge no solo por ser una herramienta capaz de procesar y generar lenguaje natural con un nivel alto de coherencia, sino también por la habilidad que tienen para escribir líneas y líneas de código, procesar una gran cantidad de archivos, lograr encontrar errores que tiempo atrás un desarrollador se tardaría más tiempo en resolver por estas razones y otras como su amplia ventana de contexto es que en los últimos años se ha visto creciente uso de los modelos grandes de lenguajes (LLMs) en ciclo de vida de desarrollo de software más específicamente en las fases de implementación y pruebas.

Sin embargo, el uso de LLMs en la ingeniería de requisitos ha evolucionado rápidamente, los primeros enfoques (2023-2024) se centraban en su uso como asistentes de análisis textual mientras que investigaciones más recientes (2025-2026) proponen arquitecturas más complejas basadas en Retrieval-Augmented Generation (RAG) y sistemas multiagente, orientadas a mejorar la precisión, trazabilidad y robustez de los resultados.

2. Metodología de revisión

La metodología que se llevó a cabo fue plantear una ecuación de búsqueda lo suficientemente acotada para que no introdujera ruido en los artículos que luego pasarían a ser parte los mencionados en este estado del arte se tuvo en cuenta una ventana de tiempo de 5 años, dicha ecuación fue:

(("requirements engineering" OR "software requirements" OR "requirements analysis") AND ("natural language processing" OR NLP OR "large language model*" OR LLM*) AND

("retrieval-augmented generation" OR RAG OR "retrieval augmented generation") AND (classification OR extraction OR tracing OR validation OR "requirements elicitation")) OR (NLP4RE) AND PUBYEAR > 2020 AND (LIMIT-TO (SUBAREA , "COMP"))+

Dicha ecuación arrojó un resultado de 217 artículos de investigación para posteriormente hacer un primer barrido de descarte de estos basándonos en el nombre del artículo y si no se lograba percibir la esencia se hacía por las palabras claves

El segundo y último filtro se hizo a través del resumen de cada artículo se trató de incluir artículos que fuesen muy alineados con el proyecto de investigación y finalmente se obtuvo un total de 17 artículos

Los artículos se leyeron y se usó la metodología de ficha documental para lograr extraer las ideas principales o más relevantes en nuestro caso.

3. Fundamentos de la Ingeniería de Requerimientos

La ingeniería de requerimientos o elicitación de requisitos es el proceso donde se capturan las necesidades de los stakeholders (personas, usuarios de interés en el sistema), formalizar este proceso de revisión permite establecer un filtro estandarizado. Dicho filtro asegura que cada especificación cumpla con los atributos normativos exigidos por la industria previniendo la propagación de defectos hacia etapas del ciclo de vida donde su corrección resulta exponencialmente más costosa.

3.1 Calidad:

La calidad de los requisitos de software no es un atributo estético de la documentación, sino una métrica crítica de viabilidad técnica y mitigación de riesgo. Se define como el grado en que las especificaciones capturan de manera fidedigna las necesidades del dominio y del problema y proporcionan una base sólida y verificable para las fases de arquitectura, codificación y pruebas [1], Un requisito de alta calidad minimiza la entropía del proyecto; los defectos introducidos en esta etapa y descubiertos durante la implementación o el despliegue tienen un costo de corrección exponencialmente mayor [2]. Por tanto, la evaluación técnica debe ser un proceso sistemático de escrutinio temprano.

3.2 Taxonomía de defectos estructurales en la especificación:

La redacción en lenguaje natural, aunque accesible para los stakeholders, introduce vulnerabilidades semánticas que dificultan la evaluación técnica. Los defectos más críticos incluyen:

- **Ambigüedad:** Ocurre cuando un requisito es susceptible a múltiples interpretaciones. Puede ser léxica, sintáctica o pragmática. Esto provoca divergencias entre lo que el cliente espera y lo que el equipo de desarrollo construye [2]
- **Inconsistencia:** Se manifiesta cuando dos o más requisitos presentan contradicciones lógicas, temporales o de restricciones operativas. Esto frecuentemente genera conflictos en la arquitectura del sistema, haciendo imposible la satisfacción simultánea de todas las especificaciones.
- **Incompletitud:** Es la ausencia de información necesaria para la implementación. Esto incluye la omisión de respuestas del sistema ante flujos alternativos o excepciones, la falta de definición de interfaces o la ignorancia de atributos de calidad (rendimiento, seguridad, disponibilidad)

3.3 Marco normativo: Estándar ISO/IEC/IEEE 29148:2018

Para mitigar la subjetividad en la evaluación, la ingeniería de software se apoya en marcos estandarizados. La norma ISO/IEC/IEEE 29148 establece directrices formales para la especificación de requisitos y define los criterios sintácticos y semánticos que determinan su validez [4]. Según el estándar, un requisito individual estructurado correctamente debe cumplir con los siguientes atributos demostrables:

- **Singularidad:** Declara una única función, propiedad o restricción, evitando la concatenación de ideas mediante conjunciones complejas.
- **Ausencia de ambigüedad:** Utiliza terminología estandarizada del glosario del proyecto, cuantificadores exactos y verbos imperativos claros (“El sistema debe”).
- **Compleitud:** Es autocontenido. Proporciona toda la información de entrada, el procesamiento esperado y las condiciones de salida necesarias para su diseño.
- **Consistencia:** Mantiene cohesión técnica y lógica con el resto del documento de especificación (SRS).
- **Verificabilidad:** Su redacción permite la derivación directa de casos de prueba finitos, con criterios de aceptación cuantificables o booleanos.
- **Trazabilidad:** Mantiene identificadores únicos que permiten rastrear su origen hacia los objetivos del negocio y su evolución hacia artefactos de código y prueba.

3.4 Sistematización de la evaluación técnica

La evaluación manual de estos atributos contra grandes volúmenes de especificaciones es propensa al error humano y a la fatiga cognitiva. Por consiguiente, la validación de las características exigidas por la norma ISO/IEC/IEEE 29148 requiere de enfoques automatizados. La aplicación de modelos de lenguaje asistido por bases de conocimiento permite contrastar semánticamente los requisitos en lenguaje natural contra las reglas ontológicas de la ingeniería de software, identificando desviaciones, ambigüedades implícitas y dependencias rotas con un nivel de precisión superior al análisis heurístico tradicional [5].

4. Enfoques de NLP para la evaluación de requisitos

4.1 Enfoques basados en LLMs

Los modelos de lenguaje de gran escala han emergido como herramientas centrales para automatizar tareas de ingeniería de requisitos. Sin embargo, su eficacia no depende únicamente de la capacidad del modelo, sino de la forma en que se les intruye. En este sentido, la ingeniería de prompts (PE) actúa como interfaz de programación que guía y controla el comportamiento del LLM, siendo fundamental para obtener resultados coherentes y reproducibles [7].

Entre las técnicas de prompting más relevantes para la ingeniería de requisitos se destacan: Chain-of-Thought (CoT), que descompone el razonamiento paso a paso mejorando la precisión en tareas complejas; Few-shot prompting, que provee ejemplos dentro del propio prompt para orientar el formato y el estilo de respuesta; y Multi-role prompting, que simula roles especializados (analista, QA, arquitecto) para obtener perspectivas diversas sobre un mismo requisito [7]. La ausencia de metodología formal para seleccionar y combinar estas técnicas representa aún una limitación significativa en el campo.

Respecto a la evaluación de calidad, investigaciones recientes han evaluado la capacidad de modelos como Llama 2-70B para verificar requisitos contra las 9 características del estándar ISO/IEC/IEEE 29148, entre ellas: singularidad, ausencia de ambigüedad, completitud y verificabilidad. Los resultados demuestran que el LLM tiende a ser más estricto que los evaluadores humanos, pero sus explicaciones son plausibles y las mejoras sugeridas resulta útiles, posicionando al modelado como un asistente eficaz para el aseguramiento de la calidad [15].

En el contexto ágil, herramientas como Epic Evaluator, desarrollada por investigadores de IBM y basada en Llama 3-70B, han demostrado que es posible estandarizar la evaluación de Épicas Ágiles mediante rúbricas estructuradas de ocho elementos (problema, métricas, historias de usuario, supuestos, requisitos, no funcionales, entre otros). Un estudio con 17 Product Manager (PM) evidenció alta aceptación de la herramienta para reducir la ambigüedad y facilitar la computación inter-equipos, aunque también identificó como principal limitación la falta de conocimiento profundo del dominio específico de cada organización por parte del modelo [13].

La elicitación de requisitos también se ha beneficiado de los LLMs. En el ámbito del entrenamiento de analistas, GPT-4 ha sido utilizado para generar guiones de entrevistas de elicitación libres de errores comunes, mediante la técnica de In-Context Learning y encadenamiento de prompts (Prompt Chaining) para evitar truncamientos por límites de tokens [11]. Por su parte, GPT-4o ha demostrado ser capaz de generar preguntas de seguimiento durante

entrevistas en tiempo real reduciendo la carga cognitiva del analista. Cuando se le instruye con un marco que describe errores comunes a evitar (mistake-guided generation), las preguntas generadas superan la calidad a las formuladas por humanos [12].

Finalmente, en el plano de la generación de especificaciones formales, SpecGen introduce un ciclo de retro alimentación conversacional entre el LLM y un verificador formal (OpenJML): los errores de verificación son enviados como feedback iterativo al modelo, que corrige sus propias salidas. Cuando esta estrategia falla, se aplica un módulo de generación basada en mutación sobre operadores sintácticos. Sobre un benchmark de 385 programas Java complejos, SpecGen superó ampliamente a técnicas clásicas de inferencia de especificaciones [21].

En el dominio de verificación de satisfactibilidad de requisitos sobre cadenas de texto, un enfoque híbrido combina LLMs con verificadores formales autogenerados (en SMT y Python). El LLM genera tanto verificadores como las soluciones, mientras que los verificadores auditan los resultados y retroalimentan al modelo en un ciclo iterativo. Este mecanismo de auto-corrección logró duplicar la precisión (F1-Score) respecto a enfoques sin retroalimentación en modelos como GPT-4o y LLama 3 [22].

4.2 Enfoques basados en RAGs

Retrieval-Augmented Genration (RAG) es un arquitectura que cambia la recuperación de información relevante desde una base de datos de conocimiento externa con la capacidad generativa de un LLM. Su flujo consta de dos etapas principales: una fase de indexación (offline), en la que los documentos se cargan, fragmentan en chunks semánticos y se almacenan en un vector store; y una fase de generación (runtime), en la que los fragmentos más relevantes se recuperan mediante búsqueda semántica y se inyectan en el prompt del LLM junto con la consulta del usuario [6].

Esta arquitectura resulta especialmente valiosa en dominios donde la privacidad y la sensibilidad de la documentación impiden el envío de requisitos a APIs externas. En el sector automotriz, se ha identificado que aunque los LLMs se emplean ampliamente en codificación, su uso para la ingeniería de requisitos es aún incipiente, en parte por restricciones de privacidad y la escasez de datasets etiquetados de dominio [6].

Un caso representativo de RAG aplicado a la gestión de procesos de negocio es el chatbot PRODIGY, desarrollado para el Grupo Hilti y basado en GPT-3.5. Mediante RAG sobre la base de conocimiento interna de la empresa, desplegado en infraestructura privada, el sistema permite a los modeladores de procesos reducir el tiempo promedio de comprensión de un proceso de 40 minutos a consultas directas en lenguaje natural. Las salidas son compatibles con BPMN Sketch Miner para la generación automática de diagramas. Los ingenieros destacan que la IA es útil solo cuando se acopla estrictamente al contexto organizacional y siempre con revisión humana (human-in-the-loop) [14].

En la industria aeroespacial, RAG ha sido empleado para derivar automáticamente requisitos de software y hardware a partir de cientos de páginas de documentos de misión y estándares regulatorios como los ECSS. El pipeline propuesto clasifica textos mediante etiquetas neuronales, recupera fragmentos normativos relevantes mediante embeddings y los inyecta en modelos como OpenAI o1 o DeepSeek-R1 para generar borradores de requisitos alineados a la normativa. Este enfoque reduce la barrera de entrada para organizaciones pequeñas en proyectos de misión crítica [17].

El sistema QAssist combina dos motores de recuperación de información (IR) con un modelo de comprensión lectora (MRC) para responder preguntas de analistas auditando Documentos de Especificación de Requisitos (SRS). Ante una pregunta, el sistema busca en el SRS y en una base de conocimiento externa de dominio (corpus de Wikipedia generados a partir de los términos clave del SRS). Evaluado sobre el dataset REQuestA (387 pares pregunta-respuesta en dominios de aeronáutica, defensa y seguridad), QAssist alcanzó un recall del 90% en localización de fragmentos dentro del SRS y del 96% en la fuente externa, con una exactitud de extracción del 84% [18].

4.3 Enfoques basados en Agentes

Los sistemas multiagente basados en LLMs (LMA) representan la frontera más avanzada en la aplicación de inteligencia artificial a la ingeniería de software. Su arquitectura típica comprende una plataforma de orquestación y múltiples agentes con roles especializados (Orquestador, Programador, Revisor, Tester) que colaboran de forma autónoma para resolver tareas complejas. En el contexto de la ingeniería de requisitos, los agentes pueden simular stakeholders, generar guiones de debate y priorizar historias de usuario, mitigando alucinaciones mediante validación cruzada entre agentes [8].

El paradigma de Debate Multi-Agente (MAD) propone que múltiples instancias de LLMs asuman roles diferenciados (debatiente a favor, debatiente en contra, juez) para discutir y refinar respuestas, emulando la colaboración humana experta. Aplicado a la clasificación de requisitos funcionales y no funcionales, el enfoque MAD supera holgadamente a los esquemas de agente único, mejorando de forma significativa el F1-Score, especialmente en la identificación de requisitos no funcionales (reducción de falsos positivos). La principal limitación reportada es el costo computacional, que puede ser hasta 33 veces superior en tokens respecto a un agente único, obligando a un balance explícito entre precisión y costo [19].

Para la derivación automática de requisitos de seguridad en vehículos autónomos, se ha propuesto un modelo RAG Multi-Agente en el que cada normativa (como ISO 26262) cuenta con un agente especializado equipado con un índice de vectores (para recuperación de hechos) y un índice de resúmenes (para comprensión global). Un Agente Principal delega las consultas a los agentes más relevantes, que procesan y refina el contexto antes de enviarlo al LLM final. Evaluado sobre 58 casos del sistema de conducción autónoma Apollo, este enfoque mejoró

notablemente la Precisión de Recuperación y la Consistencia de Respuesta frente al RAG convencional [20].

El modelo HARE-SM (Human-AI RE Sunergy Model) propone un marco de integración donde la automatización mediante IA actúa como apoyo, pero la decisión final siempre recae en el analista humano (human-in-the-loop). El modelo se apoya en cuatro pilares: validación humana y explicabilidad (XAI), mitigación de sesgos y calibración de la confianza. Aplicado en la generación de criterios de aceptación, múltiples LLMs sugieren opciones que el analista evalúa, edita y consolida, generando logs para el aprendizaje continuo [10].

5. Tareas Abordadas en NLP para la ingeniería de requerimientos

La literatura revisada aborda un conjunto diverso de tareas en las que el procesamiento de lenguaje natural se aplica a la ingeniería de requisitos. A continuación se describen las más relevantes identificadas en los artículos analizados.

Clasificación de requisitos: Consiste en categorizar automáticamente los requisitos en funcionales (RF) y no funcionales (RNF), o en subcategorías temáticas. Esta tarea ha sido abordada mediante debates multiagente que superan la precisión de los clasificadores convencionales, especialmente en la identificación de RNF donde los modelos de agente único exhiben alta tasa de falsos positivos [19]. También se han desarrollado taxonomías especializadas, como la clasificación de requisitos de privacidad en 7 categorías y 71 especificaciones técnicas derivadas de regulaciones internacionales (GDPR, ISO/IEC 29100, PDPA) [9].

Evaluación de calidad de requisitos: Comprende la detección automática de defectos como ambigüedad, incompletitud e inconsistencia, así como la verificación del cumplimiento con estándares normativos. Experimentos con Llama 2-70B han demostrado que los LLMs pueden evaluar requisitos contra características del estándar ISO/IEC/IEEE 29148, explicar sus veredictos y sugerir reformulaciones mejoradas [15]. En entornos ágiles, herramientas basadas en LLMs evalúan épicas mediante rúbricas estructuradas, aportando criterios objetivos y homogéneos [13].

Elicitación de requisitos: Abarca el apoyo a las técnicas de captura de necesidades como entrevistas y talleres. Los LLMs han sido utilizados para generar guiones de entrevistas libres de errores comunes [11] y para proveer preguntas de seguimiento en tiempo real durante sesiones de elicitación, reduciendo la carga cognitiva del analista e incrementando la calidad de la información obtenida [12].

Generación de requisitos: Implica producir borradores de especificaciones a partir de documentos de dominio, estándares regulatorios o descripciones de alto nivel. En la industria espacial, pipelines RAG han sido capaces de generar requisitos alineados a estándares ECSS a

partir de documentos de misión [17]. En modelado de procesos de negocio, chatbots basados en RAG facilitan la generación de flujos BPMN a partir de consultas en lenguaje natural [14].

Auditoria y respuesta a preguntas sobre SRS: Consiste en asistir al revisor de una especificación para resolver dudas concretas sobre el documento. El sistema QAssist combina recuperación de información y compresión lectora para localizar respuestas tanto dentro del SRS como en bases de conocimiento externas, funcionando como un buscador especializado para audiotres de requisitos [18].

Generación de especificaciones formales: Implica transformar descriptores de software en lenguaje natural o semiestructurado en especificaciones verificables (precondiciones, postcondiciones, invariantes). SpecGen realiza este proceso mediante un ciclo de retroalimentación iterativa con un verificador formal, complementado por mutación sintáctica cuando el modelo falla [21]. De forma complementaria, la verificación de satisfactibilidad de requisitos sobre cadenas de texto combina generación de verificadores en Python y SMT con un ciclo de autocorrección guiado por los resultados de dichos verificadores [22].

Modelado dirigido por modelos (MDE): La generación de artefactos estructurados (diagramas UML, modelos de clases) a partir de lenguaje natural mediante LLMs representa un área emergente. Un marco técnico de cuatro pasos; formulación del problema, representación del artefacto, arquitectura del LLM y post-procesamiento. Ha sido propuesto para guiar la adopción de LLMs en entornos de ingeniería dirigida por modelos [16].

6. Datasets y recursos

La disponibilidad de conjuntos de datos etiquetados es un factor determinante para el entrenamiento y la evaluación de modelos en el contexto de la ingeniería de requisitos asistida por NLP. La revisión de literatura revela que la escasez de datasets públicos de dominio es una de las principales barreras para el avance del campo [6].

REQuestA: Creado en el contexto del sistema QAssist, este dataset contiene 387 pares pregunta-respuesta extraídos de especificaciones de requisitos en los dominios de aeronáutica, defensa y seguridad. Constituye uno de los pocos recursos anotados públicamente orientados a la tarea de respuesta a preguntas sobre SRS en dominios especializados [18].

SpecGenBench: Desarrollado para la evaluación de SpecGen, este benchmark incluye 120 programas complejos en Java y casos de prueba derivados de SV-COMP (Software Verification Competition). Está orientado a la evaluación de técnicas de generación automática de especificaciones formales (precondiciones, postcondiciones e invariantes)[21].

PROMISE NRF: Aunque no siempre citado explícitamente en todos los artículos revisados, este dataset es ampliamente referenciado en la comunidad NLP4RE como recurso de referencia para

la clasificación de requisitos funcionales y no funcionales. Contiene 625 requisitos anotados provenientes de especificaciones reales de software [5].

Corporativo industrial privado: Varios estudios, en particular los realizados en entornos corporativos como el Grupo Hilti [14] o en la industria automotriz [6], utilizan datasets propietarios que no se publican por razones de privacidad y confidencialidad. Esta práctica limita la reproducibilidad de los experimentos y dificulta la comparación entre enfoques, lo que ha sido señalado como una brecha crítica en el campo.

En cuanto a modelos base y herramientas, los estudios revisados emplean predominantemente modelos de la familia GPT (GPT-3.5, GPT-4, GPT-4o) de OpenAI, modelos de código abierto de Meta como Llama 2-70B y Llama 3, el modelo DeepSeek-R1, y modelos de comprensión lectora como BERT y T5. Para la verificación formal, se recurre a herramientas como OpenJML y solver SMT Z3.

La gestión de vectores para RAG se apoya en frameworks como LangChain y bases de datos vectoriales como FAISS o Chroma

7. Métricas de evaluación

La evaluación de sistemas NLP aplicados a la ingeniería de requisitos requiere un conjunto heterogéneo de métricas, dados que las tareas abarcan van desde la clasificación hasta la generación de texto y la verificación formal. A continuación se describen las métricas más utilizadas en los artículos revisados.

Precisión, Recall y F1-Score: Son las métricas estándar para tareas de clasificación de requisitos (funcionales vs. No funcionales). El F1-Score, que pondera armónicamente la precisión y el recall, es la métrica más reportada ya que penaliza los desequilibrios entre clases. Estudios de clasificación mediante debate multiagente reportan mejoras sustanciales en F1 respecto a enfoques de agente único, especialmente en la clase de requisitos no funcionales [19].

Retrieval Precision (RP) y Answer Consistency (AC): Utilizadas en sistemas basados en RAG para medir, respectivamente, la relevancia de los fragmentos recuperados del corpus y la coherencia de la respuesta generada con respecto al contexto recuperado. El modelo RAG Multi-Agente para derivación de requisitos de seguridad en vehículos autónomos mejoró ambas métricas de forma significativa frente al RAG convencional [20].

GRUEN (Grammaticality, Non-redundancy, Understandability, ENgagement): Métrica automática de calidad de texto aplicada para evaluar la coherencia y naturalidad de los guiones de elicitación generados por GPT-4. Su uso permite aproximar la calidad lingüística sin necesidad de evaluadores humanos para cada instancia, aunque los expertos identificaron limitaciones en la profundidad de las respuestas ante requisitos no funcionales complejos [11],

Exactitud de extracción de respuestas: En sistemas de QA sobre SRS, se mide el porcentaje de respuestas en que el fragmento extraído es correcto. QAssist reportó una exactitud del 84% en la extracción de la respuesta final, con un recall del 90% y 96% en la localización del fragmento relevante dentro del SRs y de la base de conocimiento externa, respectivamente [18].

Evaluación por expertos humanos: Complementaria a las métricas automáticas, la evaluación humana es utilizada especialmente para juzgar la utilidad, relevancia y plausibilidad de los requisitos generados o de las preguntas de seguimiento propuestas. En el estudio de elicitación asistida por LLMs, evaluadores expertos confirmaron que las preguntas generadas con un framework de errores comunes (mistake-guided generation) superaron en calidad a las formuladas por humanos [12].

Aceptación de usuarios (User Acceptance): En estudios de caso con herramientas integradas en flujos de trabajo reales (como Epic Evaluator o PRODIGY), la aceptación de los usuarios se evalúa mediante encuestas y análisis cualitativos. Los resultados indican que la estandarización del proceso de revisión y la reducción del esfuerzo cognitivo son factores de mayor valor percibido por los usuarios [13, 14].

8. Discusión

El análisis de los artículos revisados permite identificar convergencias y tensiones entre los distintos enfoques para la automatización de la ingeniería de requisitos mediante NLP. En términos generales, se observa una evolución desde el uso de LLMs como asistentes textuales simples hacia arquitecturas más sofisticadas que combinan recuperación de información, orquestación multiagente y ciclos de retroalimentación formal.

En cuanto a los LLMs puros con ingeniería de prompts, los resultados son prometedores para tareas bien delimitadas como la clasificación de requisitos o la evaluación de calidad contra criterios normativos. Sin embargo, la ausencia de metodología formal para el diseño de prompts y la variabilidad de resultados entre modelos constituyen limitaciones estructurales [7]. Los modelos tienden a ser más estrictos que los evaluadores humanos en la aplicación de estándar ISO 29148, lo cual puede introducir falsos positivos de defectos que generen trabajo adicional para los equipos [15].

Los enfoques RAG representan el avance más consolidado para entornos industriales donde la privacidad de los requisitos es una restricción no negociable. La posibilidad de desplegar modelos en infraestructura privada y enriquecerlos con bases de conocimiento propietarias reusale parcialmente el problema de los LLMs genéricos, que carecen del contexto organizacional necesario para generar especificaciones útiles [14]. No obstante, la calidad del RAG depende críticamente de la estrategia de chunking e indexación; un RAG mal configurado recupera fragmentos irrelevantes, comprometiendo la calidad de la generación [20].

Los sistemas multiagentes ofrecen el mayor potencial para tareas complejas que requieren perspectivas múltiples o validación cruzada. El paradigma de MAD reduce alucinaciones y mejora la clasificación, pero su costo computacional lo hace inviable para grandes volúmenes de requisitos sin estrategias de optimización [19]. La integración de agentes especializados por dominio normativo, como en el modelo RAG Multi-Agente para ISO 26262, representa una dirección promisoría para dominios de misión crítica [20].

Un elemento transversal en los tres enfoques es el principio de human-in-the-loop: los estudios coinciden en que la supervisión humana es indispensable para garantizar la claridad y la pertinencia de los resultados, especialmente en contextos donde los errores tienen consecuencias graves (sistemas de seguridad crítica, regulaciones legales). El modelo HARE-SM formaliza este principio integrando explicabilidad (XAI) y calibración de la confianza como pilares del diseño [10].

Desde la perspectiva de la cobertura de tareas, la revisión de evidencia que la elicitación y la evaluación de calidad son las tareas con mayor atención investigativa, mientras que la trazabilidad automática de requisitos y la gestión de cambio permanecen como áreas poco exploradas en el conjunto de artículos analizados. La brecha entre investigación académica y la adopción industrial es notable: varios estudios reportan que la falta de datasets públicos de dominio obliga a los investigadores a trabajar con datos sintéticos o propietarios que limitan la reproducibilidad [6, 13].

9. Brechas de investigación

A partir del análisis comparativo de los artículos revisados, se identifican las siguientes brechas de investigación con relevancia directa para el proyecto de investigación:

Escasez de datasets públicos de dominio: La falta de conjuntos de datos etiquetados y públicos para la evaluación de requisitos de software en dominios específicos es la limitación más recurrente en la literatura [6, 13]. La mayoría de los estudios utilizan datos propietarios o sintéticos, lo que impide la comparación directa entre enfoques y ralentiza el progreso del campo. Se requiere el desarrollo de benchmarks estándar con anotaciones normativas (ej ISO 29128) para facilitar la evaluación rigurosa.

Ausencia de metodología formal para ingeniería de prompts en RE: aunque la ingeniería de prompts es reconocida como la interfaz de control de los LLMs, no existe actualmente una metodología estandarizada para seleccionar, combinar y evaluar técnicas de prompting (CoT, Few-Shot, Multi-role) en el contexto específico de la evaluación de requisitos [7]. La falta de este marco metodológico hace que los resultados sean difícilmente reproducibles y comparables.

Integración limitada con herramientas estándar de gestión de requisitos: La mayoría de los sistemas propuestos en la literatura son prototipos de investigación desconectados de las herramientas industriales de gestión de requisitos (como DOORS, Jira o Azure DevOps). La integración de pipelines RAG y sistemas multiagente en estos entornos es una brecha práctica cuya resolución determinaría la viabilidad de adopción industrial.

Evaluación de trazabilidad end-to-end: El seguimiento de un requisito desde su origen en los objetivos de negocio hasta los artefactos de código y prueba es uno de los atributos más valorados por el estándar ISO/IEC/IEEE 29148, pero los estudios revisados no abordan su automatización de forma integral. La derivación automática de matrices de trazabilidad a partir de LLMs y bases de conocimiento es un área casi inexplorada en el corpus analizado [5].

Generalización entre dominios y tipos de sistemas: Los modelos evaluados muestran un rendimiento fuertemente dependiente del dominio (automotriz, aeroespacial, ágil). No se han desarrollado enfoques que demuestren robustez y generalización entre dominios heterogéneos, lo que limita la aplicabilidad de los sistemas propuestos a contextos más amplios.

Balance entre automatización y control humano: Si bien el principio de human-in-the-loop es ampliamente reconocido, no existe consenso sobre el grado óptimo de automatización ni sobre los mecanismos de explicabilidad (XAI) que deben acompañar a las decisiones de los sistemas de IA en RE [10]. Esta brecha es especialmente relevante para contextos regulados donde la trazabilidad de las decisiones tiene implicaciones legales.

10. Conclusiones

Este estado del arte ha revisado 17 artículos de investigación publicados entre 2020 y 2026 sobre el uso de modelos de lenguaje de gran escala, técnicas de RAG y sistemas multiagente para la asistencia en tareas de ingeniería de requisitos de software. Los principales hallazgos se sintetizan a continuación.

Primero, el campo ha evolucionado rápidamente desde enfoques puramente textuales hacia arquitecturas híbridas que combinan recuperación semántica, generación y verificación. Los LLMs por sí solos son insuficientes para garantizar la calidad y la trazabilidad de los requisitos; su efectividad depende críticamente de estrategias de prompting estructurado, del acceso a bases de conocimiento de dominio y de mecanismos de retroalimentación formal.

Segundo, RAG se ha consolidado como el enfoque más práctico para contextos industriales, al permitir el enriquecimiento de modelos genéricos con conocimiento propietario sin comprometer la privacidad de los datos. Su combinación con agentes especializados por normativa (Agent-based RAG) representa la dirección más prometedora para dominios de misión crítica como el automotriz o el aeroespacial.

Tercero, el estándar ISO/IEC/IEEE 29148 emerge como el marco normativo de referencia para la evaluación automatizada de calidad de requisitos. Los LLMs son capaces de verificar el cumplimiento de sus atributos con explicaciones plausibles, aunque con una tendencia a mayor estrictez respecto a los evaluadores humanos, lo que sugiere que deben emplearse como asistentes y no como árbitros finales.

Cuarto, la supervisión humana (human-in-the-loop) no es un aspecto opcional sino una necesidad estructural del diseño de sistemas de IA para la ingeniería de requisitos. La explicabilidad de las decisiones del modelo y la calibración de la confianza son condiciones sine qua non para la adopción en entornos regulados.

Las brechas identificadas escasez de datasets públicos de dominio, ausencia de metodología formal de prompting, desconexión con herramientas industriales y falta de cobertura de la trazabilidad end-to-end definen el espacio de contribución del presente proeycot de investigación, orientado al desarrollo de un modelo de lenguaje asistido por base de conocimiento para evaluación técnica de requisitos de software bajo el marco del estándar ISO/IEC/IEEE 29148.

11. Referencias:

- [1] Software Requirements, 3rd Edition | Microsoft Press Store. (s/f). Recuperado el 4 de mayo de 2026, de <https://www.microsoftpressstore.com/store/software-requirements-9780735679610>
- [2] Boehm, B., & Basili, V. R. (2001). Top 10 list [software development]. Computer, 34(1), 135–137. <https://doi.org/10.1109/2.962984>
- [3] Requirements Engineering. (s/f). Recuperado el 4 de mayo de 2026, de <https://link.springer.com/book/9783662692042>
- [4] 29148-2018—ISO/IEC/IEEE International Standard—Systems and software engineering—Life cycle processes—Requirements engineering. (2018). IEEE.
- [5] Zhao, L., Alhoshan, W., Ferrari, A., Letsholo, K. J., Ajagbe, M. A., Chioasca, E.-V., & Batista-Navarro, R. T. (2020). Natural Language Processing (NLP) for Requirements Engineering: A Systematic Mapping Study (arXiv:2004.01099). arXiv. <https://doi.org/10.48550/arXiv.2004.01099>
- [6] Petrovic, N., Zolfaghari, V., Schamschurko, A., Kirchner, S., Pan, F., Wu, C., Purschke, N., Velsh, A., Lebioda, K., Song, Y., Zhang, Y., Mazur, L., & Knoll, A. (2025). Survey of GenAI for Automotive Software Development: From Requirements to Executable Code (arXiv:2507.15025). arXiv. <https://doi.org/10.48550/arXiv.2507.15025>

- [7] Huang, K., Wang, F., Huang, Y., & Arora, C. (2025). Prompt Engineering for Requirements Engineering: A Literature Review and Roadmap (arXiv:2507.07682). arXiv. <https://doi.org/10.48550/arXiv.2507.07682>
- [8] He, J., Treude, C., & Lo, D. (2025). LLM-Based Multi-Agent Systems for Software Engineering: Literature Review, Vision and the Road Ahead (arXiv:2404.04834). arXiv. <https://doi.org/10.48550/arXiv.2404.04834>
- [9] Sangaroonsilp, P., Dam, H. K., Choetkiertikul, M., Ragkhitwetsagul, C., & Ghose, A. (2023). A Taxonomy for Mining and Classifying Privacy Requirements in Issue Reports (arXiv:2101.01298). arXiv. <https://doi.org/10.48550/arXiv.2101.01298>
- [10] Abbasi, M. A., Ihantola, P., Mikkonen, T., & Mäkitalo, N. (2025). Towards Human-AI Synergy in Requirements Engineering: A Framework and Preliminary Study (arXiv:2510.25016). arXiv. <https://doi.org/10.48550/arXiv.2510.25016>
- [11] Görer, B., & Aydemir, F. B. (2024). GPT-Powered Elicitation Interview Script Generator for Requirements Engineering Training (arXiv:2406.11439). arXiv. <https://doi.org/10.48550/arXiv.2406.11439>
- [12] Shen, Y., Singhal, A., & Breaux, T. (2025). Requirements Elicitation Follow-Up Question Generation (arXiv:2507.02858). arXiv. <https://doi.org/10.48550/arXiv.2507.02858>
- [13] Geyer, W., He, J., Sarkar, D., Brachman, M., Hammond, C., Heins, J., Ashktorab, Z., Rosenberg, C., & Hill, C. (2025). A Case Study Investigating the Role of Generative AI in Quality Evaluations of Epics in Agile Software Development (arXiv:2505.07664). arXiv. <https://doi.org/10.48550/arXiv.2505.07664>
- [14] Ziche, C., & Apruzzese, G. (2024). LLM4PM: A case study on using Large Language Models for Process Modeling in Enterprise Organizations (arXiv:2407.17478). arXiv. <https://doi.org/10.48550/arXiv.2407.17478>
- [15] Lubos, S., Felfernig, A., Tran, T. N. T., Garber, D., Mansi, M. E., Erdeniz, S. P., & Le, V.-M. (2024). Leveraging LLMs for the Quality Assurance of Software Requirements (arXiv:2408.10886). arXiv. <https://doi.org/10.48550/arXiv.2408.10886>
- [16] Rocco, J. D., Ruscio, D. D., Sipio, C. D., Nguyen, P. T., & Rubei, R. (2024). On the use of Large Language Models in Model-Driven Engineering (arXiv:2410.17370). arXiv. <https://doi.org/10.48550/arXiv.2410.17370>
- [17] Arora, C., Wang, F., Tantithamthavorn, C., Aleti, A., & Kenyon, S. (2025). From Domain Documents to Requirements: Retrieval-Augmented Generation in the Space Industry (arXiv:2507.07689). arXiv. <https://doi.org/10.48550/arXiv.2507.07689>

- [18] Ezzini, S., Abualhaija, S., Arora, C., & Sabetzadeh, M. (2023). AI-based Question Answering Assistance for Analyzing Natural-language Requirements (arXiv:2302.04793). arXiv. <https://doi.org/10.48550/arXiv.2302.04793>
- [19] Oriol, M., Motger, Q., Marco, J., & Franch, X. (2025). Multi-Agent Debate Strategies to Enhance Requirements Engineering with Large Language Models (arXiv:2507.05981). arXiv. <https://doi.org/10.48550/arXiv.2507.05981>
- [20] Balu, B. V., Geissler, F., Carella, F., Zacchi, J.-V., Jiru, J., Mata, N., & Stolle, R. (2025). Towards Automated Safety Requirements Derivation Using Agent-based RAG (arXiv:2504.11243). arXiv. <https://doi.org/10.48550/arXiv.2504.11243>
- [21] Ma, L., Liu, S., Li, Y., Xie, X., & Bu, L. (2025). SpecGen: Automated Generation of Formal Program Specifications via Large Language Models (arXiv:2401.08807). arXiv. <https://doi.org/10.48550/arXiv.2401.08807>
- [22] Chen, B., Babikian, A. A., Feng, S., Varró, D., & Mussbacher, G. (2025). LLM-based Satisfiability Checking of String Requirements by Consistent Data and Checker Generation (arXiv:2506.16639). arXiv. <https://doi.org/10.48550/arXiv.2506.16639>